

Project 01 — Context and Starting Points

SuperCoders · Read Before Beginning Your Design Proposal

Version 1.3

What This Document Is

This document gives you exactly what you cannot reasonably figure out alone before starting Project 01. It covers heap memory and why it exists, why dynamic arrays need resizing, how linked lists are structured in memory, what hash functions do and why collisions happen, what Redis is and why it matters, the shape of what you are building, worked memory traces for all three structures, complexity expectations, how copying breaks when you own memory, and how to verify your environment.

Everything else — which resizing strategy to use, how to handle collisions, how to design your APIs — is yours to figure out. Those decisions are the project.

Read this document alongside your student handbook. Then begin your design proposal.

Section 1 — Why This Project Exists

Before Project 01 you have probably written programs that store data in variables and arrays. Those programs worked, but they relied on the language or runtime managing memory for you. You declared a variable and it appeared. The program ended and it disappeared.

Real systems do not work this way.

A database stores records while the program is running. A web crawler stores pages as it discovers them. An AI system stores embeddings for millions of documents. None of these can know in advance how much memory they need. They must request memory as they go, use it, and release it when finished.

This is called manual memory management. It is one of the most important skills in systems engineering. It is also where most bugs come from.

Project 01 teaches you to manage memory directly. You will allocate memory, use it, and free it yourself. The compiler will not help you. The runtime will not help you. If you forget to free memory it leaks. If you free it twice the program crashes. If you use it after freeing it the behavior is undefined.

This is not comfortable. It is necessary. Every system you build in Projects 02 through 05 depends on getting this right.

Forbidden Libraries

The purpose of this project is to build these structures by hand. The structures you are implementing already exist in the C++ Standard Library — and using those versions defeats the entire learning objective. During Project 01 you may **not** use any of the following inside your Collections Library:

- `std::vector`
- `std::list`
- `std::deque`
- `std::map`
- `std::unordered_map`
- `std::stack`, `std::queue`, `std::set`
- Any third-party container library

Permitted: `new`, `delete`, `string`, `iostream`, `fstream`, `sstream`.

Using an STL container internally — even if it satisfies an interface requirement — will result in rejection during code review. There is exactly one narrow exception: STL containers may appear in your **benchmark and test-harness code**, where `std::vector` and `std::unordered_map` are used only as comparison baselines for the Performance Report. They must never appear in the library implementation itself.

Section 2 — Stack Memory and Heap Memory

There are two places your program stores data. Understanding the difference between them is the foundation of everything else in this project.

Stack memory is managed automatically. When you declare a variable inside a function it lives on the stack. When the function returns the variable is destroyed.

```
cpp
void example() {
    int x = 5;      // lives on the stack
    int arr[10];    // also on the stack
}
// x and arr are gone when example() returns
```

Stack memory is fast and automatic. Its limitation is that it has a fixed size determined at compile time. You cannot create a stack array whose size you only know at runtime. You cannot return a stack array from a function and expect it to still exist.

Heap memory is managed manually. You request it with `new` and release it with `delete`. It persists until you explicitly free it — the function returning does not destroy it.

```
cpp
void example() {
    int* p = new int(5); // lives on the heap
    int* arr = new int[10]; // heap array
```

```

// p and arr still exist after example() returns
// but you are now responsible for deleting them

delete p;
delete[] arr;
}

```

The heap is where all three of your data structures live. `DynamicArray`, `LinkedList`, and `HashMap` all allocate memory on the heap at runtime because their sizes are not known at compile time.

The critical rule:

Every `new` must be paired with a `delete`. Every `new[ ]` must be paired with a `delete[ ]`. No exceptions.

```

cpp
int* x = new int(5);    → delete x;
int* arr = new int[100]; → delete[] arr;
Node* n = new Node();   → delete n;

```

If you call `new` without `delete` the memory is never returned to the operating system. The program appears to work but leaks memory with every allocation. This is a memory leak. Hidden Requirement C requires a clean valgrind run with zero leaks. The only way to pass it is to pair every `new` with a `delete` throughout your entire codebase.

Section 3 — Pointers

Heap memory is accessed through pointers. A pointer is a variable that stores a memory address rather than a value.

```

cpp
int x = 5;    // x stores the value 5
int* p = &x;  // p stores the address of x

cout << x;    // prints 5
cout << p;    // prints an address like 0x7ffd...
cout << *p;   // prints 5 — dereferences the pointer

```

When you allocate heap memory, `new` returns a pointer to the allocated block.

```

cpp
int* arr = new int[5];
// arr points to the first element of a heap array

arr[0] = 10;    // access by index
arr[1] = 20;
*(arr + 2) = 30; // same as arr[2] = 30

```

The most common pointer bugs:

cpp

// Bug 1 — Dangling pointer

```
int* p = new int(5);
```

```
delete p;
```

```
*p = 10; // undefined behavior — p points to freed memory
```

// Bug 2 — Memory leak

```
int* p = new int(5);
```

```
p = new int(10); // original allocation is lost — leaked
```

// Bug 3 — Double free

```
int* p = new int(5);
```

```
delete p;
```

```
delete p; // crash — freeing already freed memory
```

// Bug 4 — Null dereference

```
int* p = nullptr;
```

```
*p = 5; // crash — dereferencing null
```

You will encounter all four of these bugs during this project. Knowing their names and symptoms is the first step to diagnosing them quickly.

Section 4 — Why Dynamic Arrays Need Resizing

A `DynamicArray` is a growable sequence container. The key word is growable. Here is why growth requires special handling.

When you create an array in C++ you must specify its size:

cpp

```
int arr[10]; // fixed stack array
```

```
int* arr = new int[10]; // fixed heap array
```

Once created the array cannot grow. If you need to store an 11th element you cannot simply extend it — the memory after position 9 may be occupied by something else.

The solution is to allocate a new larger array, copy the old elements over, and delete the old array.

cpp

```
// Current state: array of size 4, all slots filled
```

```
// [10, 20, 30, 40] capacity=4, size=4
```

```
// Need to append 50.
```

```
// Step 1: allocate new array with more space
```

```
int* newArr = new int[8]; // doubled capacity
```

```
// Step 2: copy old elements
for (int i = 0; i < 4; i++) {
    newArr[i] = arr[i];
}

// Step 3: delete old array
delete[] arr;

// Step 4: point to new array
arr = newArr;
capacity = 8;

// Step 5: append new element
arr[4] = 50;
size = 5;
```

Two important concepts: size and capacity.

Size is how many elements are currently stored.

Capacity is how many elements the underlying array can hold.

```
arr = [10, 20, 30, 40, _, _, _, _]
      size=4          capacity=8
```

These are different numbers. Your DynamicArray must track both.

Why double the capacity rather than add one slot?

If you add one slot on every append, every single append triggers a resize. Appending N elements requires N resizes and approximately N^2 copy operations total. That is expensive.

If you double the capacity on every resize, the number of resizes is logarithmic. Appending N elements requires approximately $\log_2(N)$ resizes and approximately $2N$ copy operations total. That is much cheaper.

This is called amortized $O(1)$ append. Each individual append may occasionally trigger an expensive resize, but averaged over many appends the cost per append is constant. This is the right answer to "why double?" and you will be asked it during your oral defense.

A worked trace:

Start: `arr=[], size=0, capacity=1`

```
append(10):
    size < capacity (0 < 1). No resize.
    arr=[10], size=1, capacity=1
```

```
append(20):
```

```
size == capacity (1 == 1). Resize.  
new capacity = 2  
arr=[10,20], size=2, capacity=2
```

```
append(30):  
size == capacity (2 == 2). Resize.  
new capacity = 4  
arr=[10,20,30,_,], size=3, capacity=4
```

```
append(40):  
size < capacity (3 < 4). No resize.  
arr=[10,20,30,40], size=4, capacity=4
```

```
append(50):  
size == capacity (4 == 4). Resize.  
new capacity = 8  
arr=[10,20,30,40,50,_,_,_], size=5, capacity=8
```

After 5 appends there have been 3 resizes. After 8 appends there will still be 3 resizes — the 8th append fills capacity exactly but does not trigger a new one. After 16 appends there will be 4 resizes; the next resize does not happen until the 17th append. The resize count grows logarithmically while the element count grows linearly. That is the efficiency of doubling.

Section 5 — How Linked Lists Are Structured in Memory

A LinkedList is a chain of nodes. Each node is a separate heap allocation. Each node contains two things: a value and a pointer to the next node.

This is fundamentally different from an array. In an array all elements are stored contiguously. In a linked list the elements can be anywhere in heap memory. They are connected only by pointers.

What a node looks like:

```
cpp  
struct Node {  
    int value;  
    Node* next;  
};
```

What three nodes look like in memory:

An array stores 10, 20, 30 like this:

```
Address: 1000 1004 1008  
Value:  [10] [20] [30]
```

Contiguous. Each element 4 bytes after the previous.

A linked list stores 10, 20, 30 like this:

Address 1000: value=10 next → 2048

Address 2048: value=20 next → 3104

Address 3104: value=30 next → null

Not contiguous. Three separate heap allocations connected only by pointers.

A worked trace — insertFront:

Start: head = null

insertFront(30):

Allocate Node{30, null}

head = Node{30, null}

head → [30|null]

insertFront(20):

Allocate Node{20, null}

Node{20}.next = head // points to Node{30}

head = Node{20}

head → [20|→] → [30|null]

insertFront(10):

Allocate Node{10, null}

Node{10}.next = head // points to Node{20}

head = Node{10}

head → [10|→] → [20|→] → [30|null]

After three insertions there are three separate heap allocations connected by pointers. The head pointer is the only way to reach the list. If you lose the head pointer you lose the entire list and cannot free any of the nodes. That is a memory leak.

Deleting a node — order matters:

Before: head → [10|→] → [20|→] → [30|null]

deleteFront():

Step 1: save head in temp: temp = head

Step 2: move head forward: head = head->next

Step 3: delete saved node: delete temp

After: head → [20|→] → [30|null]

temp → freed

The order matters critically. If you delete head before saving head->next you lose the pointer to the rest of the list. That is a classic bug. You will be shown this exact bug during your oral defense.

Why Linked Lists Exist

A fair question: if arrays give $O(1)$ indexing and far better cache performance, why use a linked list at all? Because each structure is good at a different thing.

Structure	Strength	Weakness
DynamicArray	$O(1)$ indexing; contiguous and cache-friendly	$O(n)$ to insert/remove anywhere but the end — later elements must shift
LinkedList	$O(1)$ insert/delete <i>once you already hold a pointer to the position</i>	$O(n)$ to reach an arbitrary position; no random access; poor cache locality
HashMap	$O(1)$ average lookup by key	No ordering; cost of hashing and collision handling

The linked list's advantage is specific, and it is easy to state wrong. Inserting into the middle of a DynamicArray is $O(n)$ *even when you know the index*, because every later element must move over to make room. Inserting into a LinkedList is $O(1)$ — but only once you are already holding a pointer to the node you are inserting after. Reaching that node from the head is itself $O(n)$, which is why Section 8 lists `insert(index)` as $O(n)$ for both structures.

So the linked list wins when you are *already positioned* at the spot: repeatedly inserting and deleting at the front, or splicing in nodes you already hold references to, without shifting anything. This is exactly why the Linux kernel uses linked lists for structures whose elements are constantly added and removed from known positions — and why your HashMap's separate-chaining buckets are linked lists. A new collision can be inserted at the head of its chain in $O(1)$, because you already hold the head.

Section 6 — What a Hash Function Does and Why Collisions Happen

A HashMap stores key-value pairs and supports fast lookup. The speed comes from a hash function that converts any key into an array index.

The basic idea:

Instead of searching through every stored key to find the one you want ($O(n)$), you compute where it should be stored ($O(1)$) and look directly there.

cpp

```
int index = hashFunction("name"); // returns some integer
buckets[index] = "Arun";          // store at that position
```

```
int index2 = hashFunction("name"); // same key = same index
```

```
string value = buckets[index2]; // retrieve: "Arun"
```

The hash function always returns the same index for the same key. That is what makes retrieval possible.

What a simple hash function looks like:

```
cpp
int hash(string key, int tableSize) {
    int sum = 0;
    for (char c : key) {
        sum += c; // add ASCII value of each character
    }
    return sum % tableSize;
}
```

For a table of size 8:

```
hash("name") = (110+97+109+101) % 8 = 417 % 8 = 1
hash("city") = (99+105+116+121) % 8 = 441 % 8 = 1
hash("age") = (97+103+101) % 8 = 301 % 8 = 5
```

Collisions are normal and expected.

"name" and "city" both hash to index 1. This is a collision. It is not a bug. It is a mathematical inevitability — you are mapping a potentially infinite set of keys to a finite array. Some keys will always map to the same slot.

A HashMap that does not handle collisions will overwrite "name" when "city" is stored. That is a broken HashMap.

Collision handling is a design problem.

Strategy A — Separate Chaining: each bucket holds a linked list. When two keys hash to the same bucket both are stored in that bucket's list.

```
Bucket 1: [name:Arun] → [city:Chandigarh] → null
Bucket 5: [age:25] → null
```

Strategy B — Linear Probing: when a collision occurs look at the next slot. If empty store there. Lookup follows the same probe sequence.

```
Bucket 1: name:Arun
Bucket 2: city:Chandigarh (probed from bucket 1)
Bucket 5: age:25
```

Both strategies work. Both have tradeoffs. Your design proposal must state which you chose and why.

What is a load factor and why does it matter?

load factor = number of elements / number of buckets

As the load factor increases collisions become more frequent. Most HashMap implementations rehash — allocate a larger bucket array and reinsert all existing pairs — when the load factor exceeds a threshold such as 0.7. The threshold and new size are your design decisions.

A worked trace — separate chaining, table size 4:

Start: buckets = [null, null, null, null]

```
set("name", "Arun"):
  hash("name") % 4 = 417 % 4 = 1
  buckets[1] = Node{"name":"Arun", next=null}
```

[null, [name:Arun], null, null]

```
set("age", "25"):
  hash("age") % 4 = 301 % 4 = 1 ← collision
  append to buckets[1]
```

[null, [name:Arun]→[age:25], null, null]

```
set("job", "Engineer"):
  hash("job") % 4 = 315 % 4 = 3
  buckets[3] = Node{"job":"Engineer", next=null}
```

[null, [name:Arun]→[age:25], null, [job:Engineer]]

```
get("age"):
  hash("age") % 4 = 1
  traverse bucket 1:
    node 1: key="name" — no match
    node 2: key="age" — match
  return "25"
```

The lookup for "age" required traversing two nodes. With more collisions chains grow longer and lookup degrades toward $O(n)$. This is why load factor matters and why rehashing is necessary.

A Worked Rehash

Students new to hash tables often assume rehashing means "copy the buckets across." It does not. When the table grows, **every key must be hashed again against the new table size**, because `key % oldSize` and `key % newSize` are usually different numbers. Copying buckets unchanged would leave keys in the wrong place and break every future lookup.

Start: 4 buckets, two keys stored, both landing in bucket 1:

```
hash("name") % 4 = 417 % 4 = 1
hash("age") % 4 = 301 % 4 = 1 ← collision, chained in bucket 1
```

```
[null, [name:Arun]→[age:25], null, null]
```

The load factor threshold is exceeded. Grow to 8 buckets and re-insert every existing pair by recomputing its index against the new size:

new buckets = 8

```
hash("name") % 8 = 417 % 8 = 1
hash("age") % 8 = 301 % 8 = 5 ← now a different bucket
```

```
[null, [name:Arun], null, null, null, [age:25], null, null]
```

The collision that existed at size 4 is gone at size 8 — "name" and "age" now occupy different buckets. That only happens because each key was rehashed. Had you copied bucket 1 across unchanged, "age" would still be chained behind "name", and a `get("age")` after the resize might never find it. Rehashing is $O(n)$ precisely because it must touch and recompute every stored pair.

Section 7 — What Redis Is and Why It Matters

Phase 4 asks you to build Redis Lite. Before building it you should know what Redis is and why it exists.

Redis is a real production system used by Twitter, GitHub, Stack Overflow, Airbnb, and thousands of other companies. It stands for Remote Dictionary Server. It is an in-memory key-value store — it stores pairs of keys and values in memory and supports fast retrieval by key.

You use Redis when you need to store and retrieve data faster than a database can provide. A database stores data on disk. Reading from disk takes milliseconds. Redis stores data in memory. Reading from memory takes microseconds — a thousand times faster.

Common uses:

Cache a user session:

```
SET session:user123 "logged_in_at:14:30"
```

Store a counter:

```
SET page_views "14382"
```

Cache an expensive query:

```
SET top_products "[product1, product2, ...]"
```

Redis Lite is NOT:

- Not a network server

- Not multi-client
- Not persistent across restarts
- Not compatible with the real Redis protocol

If you look up Redis and see TCP servers, client libraries, and persistence layers — that is the real Redis. Redis Lite is none of those things. Building any of them is unnecessary complexity that will consume weeks without adding to what is being evaluated.

Redis Lite IS:

- A command-line program
- Single user, single session
- In-memory only — data is lost when the program exits
- Backed entirely by your HashMap
- Tested by typing commands at a terminal

Redis Lite is built on a HashMap. The commands SET, GET, DEL, and EXISTS map directly to the set, get, remove, and contains operations of your HashMap. When you complete Phase 4 you will have built a simplified but architecturally accurate version of a system that runs in production at global scale. The concepts are the same. The scale is different.

Section 8 — Complexity Reference

This table shows the expected complexity of every major operation in your three data structures. Use it to verify your implementation is on track — if your `DynamicArray.get()` is taking longer than $O(1)$ something is wrong in your design.

Structure	Operation	Expected Complexity	Why
DynamicArray	append	$O(1)$ amortized	Occasional resize, cost spread across many appends
DynamicArray	get(index)	$O(1)$	Direct index calculation — <code>arr[i]</code> is instant
DynamicArray	insert(index)	$O(n)$	Must shift elements to make room
DynamicArray	remove(index)	$O(n)$	Must shift elements to fill gap
LinkedList	insertFront	$O(1)$	Just reassign the head pointer
LinkedList	deleteFront	$O(1)$	Just move head forward and delete old head
LinkedList	search	$O(n)$	Must traverse from head — no shortcut

LinkedList	insert(index)	$O(n)$	Must traverse to find position first
HashMap	set	$O(1)$ average	Hash function computes position directly
HashMap	get	$O(1)$ average	Same — hash to position, retrieve there
HashMap	remove	$O(1)$ average	Hash to position, remove from chain or slot
HashMap	rehash	$O(n)$	Must reinsert every existing element

What "average" means for HashMap:

$O(1)$ average assumes a good hash function and a reasonable load factor. If your hash function maps every key to the same bucket, all operations degrade to $O(n)$ because every lookup traverses the entire chain. A good hash function distributes keys evenly. Testing with many different keys and checking that chains remain short is part of your testing requirement.

What "amortized" means for DynamicArray:

A single append that triggers a resize is $O(n)$ — it copies all existing elements. But resizes happen rarely enough that averaged over N appends the cost per append is $O(1)$. If you are asked "what is the complexity of append?" the correct answer is " $O(1)$ amortized" with an explanation of why. You will be asked this during your oral defense.

How to use this table:

If your implementation's performance does not match these expectations, there is a design error. Common causes:

DynamicArray get is slow:

→ Are you searching instead of indexing?

LinkedList insertFront is slow:

→ Are you traversing to the end to insert?

HashMap get is slow:

→ Is your hash function distributing keys poorly?

→ Is your load factor too high — too many collisions?

Section 9 — Copying Objects That Own Memory

This section covers one of the most common and confusing bugs you will encounter in this project. Read it carefully before you write your first class.

The problem: shallow copy.

In C++, when you copy an object the compiler's default behavior is to copy each member variable. For simple types like `int` and `double` this is correct. For pointers it is dangerous.

Consider this:

```
cpp
class DynamicArray {
    int* arr;
    int size;
    int capacity;
};

DynamicArray a;
a.append(10);
a.append(20);

DynamicArray b = a; // copy
```

After this copy, `a.arr` and `b.arr` both point to the same underlying heap array. They share memory.

```
a.arr —→ [10, 20, _, _]
           ↑
b.arr —————
```

This is called a shallow copy. The pointer was copied but the memory it points to was not.

Why shallow copy causes crashes.

When `a` goes out of scope its destructor runs and deletes the array. Now `b.arr` is a dangling pointer — it points to freed memory. When `b` goes out of scope its destructor also tries to delete the same array. This is a double free. The program crashes.

`a` goes out of scope:

`delete[] a.arr` ← frees the array. correct.

`b` goes out of scope:

`delete[] b.arr` ← tries to free already-freed memory. CRASH.

Valgrind will report this as an "Invalid free" error. The crash may happen immediately or may appear to work correctly and then crash unpredictably later. Both outcomes are caused by the same root problem.

The solution: deep copy.

A deep copy allocates new memory and copies the contents rather than copying the pointer.

```
a.arr —→ [10, 20, _, _]
```

```
b.arr —→ [10, 20, _, _] ← separate allocation, same contents
```

Now a and b each own their own memory. Deleting one does not affect the other.

The Rule of Three.

In C++, if your class manages heap memory — meaning it has a destructor that calls delete or delete[] — you almost certainly need to define all three of the following:

1. Destructor — clean up when object is destroyed
2. Copy constructor — define what copying means
3. Assignment operator — define what assignment means

This is called the Rule of Three. If you define any one of these you almost certainly need all three.

```
cpp
class DynamicArray {
public:
    ~DynamicArray();           // destructor
    DynamicArray(const DynamicArray& other); // copy constructor
    DynamicArray& operator=(const DynamicArray& other); // assignment
};
```

You are not required to implement all three for Phase 1.

Hidden Requirements A and B in this project specifically ask for the copy constructor and assignment operator. They are released after your initial submission precisely because implementing them requires understanding the problem first. This section gives you that understanding.

When you encounter a crash that says "double free" or valgrind reports "Invalid free / delete / delete[]" — return to this section. Shallow copy is almost certainly the cause.

A simple rule to avoid the problem during development:

Until you implement deep copy, do not copy your objects. Pass them by reference or by pointer. Do not assign one to another.

```
cpp
// Safe — passes by reference, no copy made
void process(DynamicArray& arr) { ... }

// Unsafe — triggers shallow copy
void process(DynamicArray arr) { ... }
```

Section 10 — Expected Component Interfaces

Your project has four major components. Here are the boundaries between them — not an implementation, just the shape of what each component is responsible for.

This is one possible decomposition. You may organize your code differently if you can justify the design. What matters is that each component has a clear responsibility and a defined interface.

cpp

```
// — DynamicArray —————  
// Growable sequence container.  
// You decide: resizing strategy, initial capacity.  
// You decide: how to handle out-of-bounds access.
```

```
class DynamicArray {  
public:  
    void append(int value);  
    void insert(int index, int value);  
    void remove(int index);  
    int get(int index);  
    int size();  
    int capacity();  
  
    // Rule of Three — implement for Hidden Requirements A and B  
    ~DynamicArray();  
    DynamicArray(const DynamicArray& other);  
    DynamicArray& operator=(const DynamicArray& other);  
};
```

```
// — LinkedList —————  
// Chain of heap-allocated nodes.  
// You decide: singly or doubly linked.  
// You decide: how to handle empty list operations.
```

```
class LinkedList {  
public:  
    void insertFront(int value);  
    void deleteFront();  
    void insert(int index, int value);  
    bool search(int value);  
    int size();  
    void print();  
  
    // Rule of Three  
    ~LinkedList();  
    LinkedList(const LinkedList& other);  
    LinkedList& operator=(const LinkedList& other);  
};
```

```
// — HashMap —————  
// Key-value store with O(1) average lookup.  
// You decide: collision strategy.
```

// You decide: load factor threshold and resize factor.

```
class HashMap {
public:
    void set(string key, string value);
    string get(string key);
    bool exists(string key);
    void remove(string key);
    int size();
    float loadFactor();

    // Rule of Three
    ~HashMap();
    HashMap(const HashMap& other);
    HashMap& operator=(const HashMap& other);
};
```

```
// — RedisCLI —————
// Command-line interface to your HashMap.
// Parses user input and calls HashMap operations.
// No networking. No persistence. Command line only.
```

```
class RedisCLI {
public:
    void run();

private:
    HashMap store;

    void handleSet(string key, string value);
    void handleGet(string key);
    void handleDel(string key);
    void handleExists(string key);
    void handleCount();
    void handleClear();
};
```

What is left for you to decide:

These interfaces define what each component does. They do not define how.

- What is the initial capacity of your DynamicArray?
- When exactly does it resize and by how much?
- Is your LinkedList singly or doubly linked?
- What data structure backs your HashMap buckets?
- What is your load factor threshold?
- How does your HashMap handle a get() call for a key that does not exist?

Every one of those decisions belongs in your design proposal. The interfaces above are the boundaries. What happens inside each boundary is your engineering work.

You are not required to use these exact class names or signatures. If you have a better decomposition and can justify it, use it.

Section 11 — How Redis Lite Uses Your HashMap

You are building the HashMap now. Redis Lite will use it in Phase 4. Understanding exactly how Redis Lite will call your HashMap changes how you design it.

```
cpp
// This is how Redis Lite will use your HashMap.
// Design your HashMap so this works correctly.

HashMap store;

// SET name Arun
store.set("name", "Arun");

// SET city Chandigarh
store.set("city", "Chandigarh");

// GET name → prints "Arun"
string value = store.get("name");
cout << value << endl;

// EXISTS city → prints 1 (true)
bool found = store.exists("city");
cout << found << endl;

// DEL name
store.remove("name");

// EXISTS name → prints 0 (false)
bool gone = store.exists("name");
cout << gone << endl;

// COUNT → prints 1
cout << store.size() << endl;

// SET name Arun (again)
// set() must overwrite if key exists
store.set("name", "Arun");
store.set("name", "Vikram"); // overwrites "Arun"
cout << store.get("name"); // must print "Vikram"
```

Reading this you can see exactly what your HashMap must support:

- set() must overwrite the value if the key already exists
- get() must return the value for an existing key — what does it return for a missing key? That is your design decision — document it
- exists() must return true or false — not crash, not return garbage
- remove() must actually delete the key — exists() must return false afterward
- size() must return the count of currently stored keys — not the bucket count, not the capacity

Design your HashMap to support all of these correctly before you build Redis Lite. Redis Lite is only as correct as the HashMap underneath it.

Section 12 — Memory Leak Detection with Valgrind

Hidden Requirement C requires a clean valgrind run. Valgrind is a tool that runs your program and reports every memory error and every leaked allocation.

Running valgrind:

```
bash
# Compile with debug symbols
g++ -g -o myprogram main.cpp

# Run under valgrind
valgrind --leak-check=full ./myprogram
```

What a clean run looks like:

```
==12345== HEAP SUMMARY:
==12345==  in use at exit: 0 bytes in 0 blocks
==12345==  total heap usage: 42 allocs, 42 frees
==12345==
==12345== All heap blocks were freed -- no leaks are possible
==12345==
==12345== ERROR SUMMARY: 0 errors from 0 contexts
```

The critical lines: "in use at exit: 0 bytes", "All heap blocks were freed", "0 errors." Every new was paired with a delete. No dangling pointer access. No double frees.

What a leaking run looks like:

```
==12345== HEAP SUMMARY:
==12345==  in use at exit: 40 bytes in 5 blocks
==12345==  total heap usage: 42 allocs, 37 frees
==12345==
==12345== 40 bytes in 5 blocks are definitely lost
==12345==    at 0x4C2FB0F: malloc
==12345==    by 0x10891: LinkedList::insertFront
==12345==                (linkedlist.cpp:23)
==12345==    by 0x10934: main (main.cpp:15)
```

```
==12345==  
==12345== ERROR SUMMARY: 0 errors from 0 contexts
```

This tells you: 5 nodes were allocated in insertFront() and never deleted. The file and line number point directly to the allocation site.

What a double free looks like:

```
==12345== Invalid free() / delete / delete[]  
==12345==   at 0x4C30D3B: operator delete[]  
==12345==   by 0x108A2: DynamicArray::~~DynamicArray  
==12345==      (dynamicarray.cpp:15)  
==12345==  
==12345== Address 0x5204e80 is 0 bytes inside a block  
==12345==   of size 40 free'd  
==12345==   by 0x108A2: DynamicArray::~~DynamicArray  
==12345==      (dynamicarray.cpp:15)
```

This is the shallow copy crash described in Section 9. Two destructors tried to delete the same array. The fix is deep copy.

The most common sources of leaks in this project:

LinkedList nodes not deleted when the list is destroyed. DynamicArray internal buffer not deleted in the destructor. HashMap bucket chain nodes not deleted during clear() or destruction. Forgetting to implement destructors entirely.

Adding destructors:

Every class that allocates heap memory must have a destructor that frees it.

```
cpp  
LinkedList::~~LinkedList() {  
    Node* current = head;  
    while (current != nullptr) {  
        Node* next = current->next;  
        delete current;  
        current = next;  
    }  
    head = nullptr;  
}
```

Section 13 — A Warning About Common Bugs

You will encounter bugs in this project that you have not encountered before. Here are the most common and how to recognize them.

Segmentation fault:

Segmentation fault (core dumped)

Your program tried to access memory it does not own. Common causes: dereferencing a null pointer, accessing an array out of bounds, using memory after deleting it. Run under valgrind — it will point to the exact line.

Infinite loop in traversal:

Your linked list traversal never terminates. Common cause: the last node's next pointer was not set to nullptr. The traversal loops back to an earlier node.

Incorrect size after delete:

You delete an element but forget to decrement the size counter. Subsequent operations use the wrong size and access invalid memory.

HashMap returning wrong value:

You retrieve from bucket[hash(key)] but find the wrong value. Common cause: stored in the wrong bucket, collision resolution is incorrect, or hash function is not deterministic.

Off-by-one in DynamicArray:

You access arr[size] instead of arr[size-1] for the last element. Or you resize when size > capacity instead of size == capacity, missing the resize by one element.

Double free from shallow copy:

You copy a data structure object and both copies try to delete the same memory. See Section 9. The fix is deep copy via copy constructor and assignment operator.

Section 14 — Environment Setup

Before writing any data structure code verify that your development environment compiles and links correctly.

```
cpp
// test_environment.cpp
// Run this first.
// If it prints success your environment is ready.

#include <iostream>
#include <string>
using namespace std;

int main() {
    cout << "Testing environment..." << endl;

    // Test heap allocation
    int* heapInt = new int(42);
    if (*heapInt != 42) {
        cout << "FAIL: heap allocation." << endl;
```

```

        return 1;
    }
    delete heapInt;
    cout << "Heap allocation: OK" << endl;

    // Test heap array
    int* heapArr = new int[5];
    for (int i = 0; i < 5; i++) heapArr[i] = i * 10;
    if (heapArr[3] != 30) {
        cout << "FAIL: heap array." << endl;
        return 1;
    }
    delete[] heapArr;
    cout << "Heap array: OK" << endl;

    // Test struct with pointer
    struct Node {
        int value;
        Node* next;
    };
    Node* n1 = new Node{10, nullptr};
    Node* n2 = new Node{20, nullptr};
    n1->next = n2;
    if (n1->next->value != 20) {
        cout << "FAIL: linked nodes." << endl;
        return 1;
    }
    delete n2;
    delete n1;
    cout << "Linked nodes: OK" << endl;

    // Test string operations
    string key = "name";
    string value = "Arun";
    if (key.length() != 4) {
        cout << "FAIL: string operations." << endl;
        return 1;
    }
    cout << "String operations: OK" << endl;

    cout << endl;
    cout << "SUCCESS: environment fully ready."
        << endl;
    cout << "You may begin your design proposal."
        << endl;

    return 0;
}

```

If any test fails resolve the environment issue before starting the project. Ask your instructor if this test does not compile.

Section 15 — What You Must Design Yourself

The following are not given to you. They are your decisions. They must be justified in your design proposal.

DynamicArray resizing strategy. When does the array resize? By how much? What is the initial capacity? Does capacity ever shrink when elements are removed?

DynamicArray error handling. What does `get()` return when the index is out of bounds? An exception, a sentinel value, or something else?

LinkedList structure. Singly or doubly linked? What is the tradeoff? How does your choice affect insertion and deletion at arbitrary positions?

LinkedList memory management. How do you ensure every node is deleted when the list is destroyed? What happens when you delete a node in the middle?

HashMap collision strategy. Separate chaining or linear probing? What are the tradeoffs? Which fits your use case better?

HashMap load factor. At what load factor do you rehash? How many buckets after rehashing? What happens to all existing key-value pairs during a rehash?

HashMap `get()` for missing keys. What does your `get()` return when the key does not exist? An empty string, a sentinel value, or an exception? Redis Lite must handle this case correctly — document your choice.

HashMap hash function. How do you convert a string key into a bucket index? What makes a good hash function? What makes a bad one?

Rule of Three. How will your destructors free all allocated memory? How will your copy constructors perform deep copies? Required for Hidden Requirements A and B.

Expected Effort

Students often struggle to judge the scope of a project like this. The estimates below are rough planning guides — not targets, and not benchmarks. Wide variation is normal, and taking longer than a range does not mean you are doing it wrong.

Component	Rough effort
DynamicArray	4–6 hours
LinkedList	4–6 hours
HashMap	8–12 hours

Redis Lite 4–6 hours

Testing & debugging 8–10 hours

Typical completion time is roughly 30–40 focused hours spread across the four weeks. The HashMap is deliberately the largest single piece — the hash function, collision handling, and rehashing are where most of the design difficulty lives. Redis Lite is comparatively small because it is a thin command layer over a HashMap you have already built and tested. If one component is taking far longer than its range, that is usually a signal to revisit a design decision rather than to keep pushing on the implementation.

Section 16 — One Last Thing

The three data structures you are about to build from scratch — dynamic arrays, linked lists, and hash tables — are among the most important data structures in computer science. They appear in:

The Linux kernel

→ linked lists for process scheduling

Every major database

→ hash tables for indexing

Every major programming language runtime

→ dynamic arrays for list types

Every search engine

→ hash tables for inverted indexes

Every AI system

→ hash tables for token vocabularies

→ dynamic arrays for embeddings

You are not building toy implementations to pass an assignment. You are building working versions of structures that power real systems. The understanding you develop here — of memory, of performance tradeoffs, of what happens when design decisions meet scale — is the foundation of everything that follows in the SuperCoders curriculum.

When you encounter a design problem — and you will — the right question is not "what is the correct answer?" The right question is "what are the tradeoffs?" Every design decision in a real system involves tradeoffs. Your job is to make explicit choices and defend them.

That is what engineers do.

Now read your student handbook and begin your design proposal.

